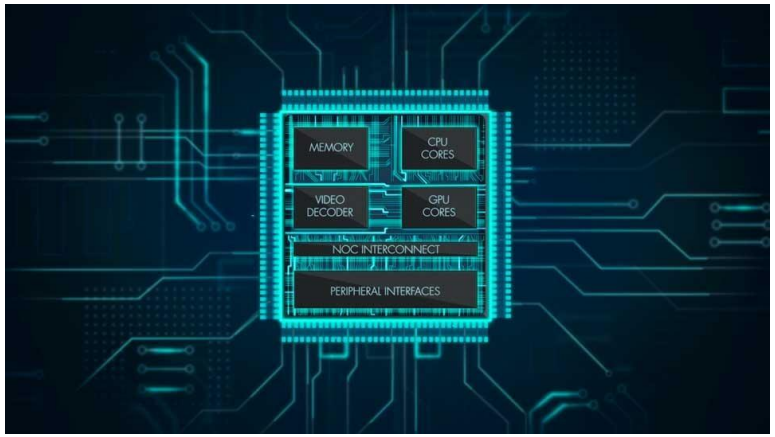
Digital System Design

Basic Practice 1



Date : 2026. 03. 26.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

Contents

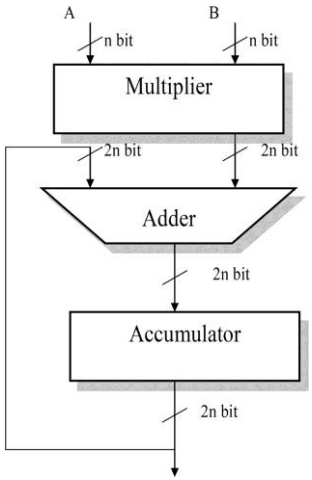
- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

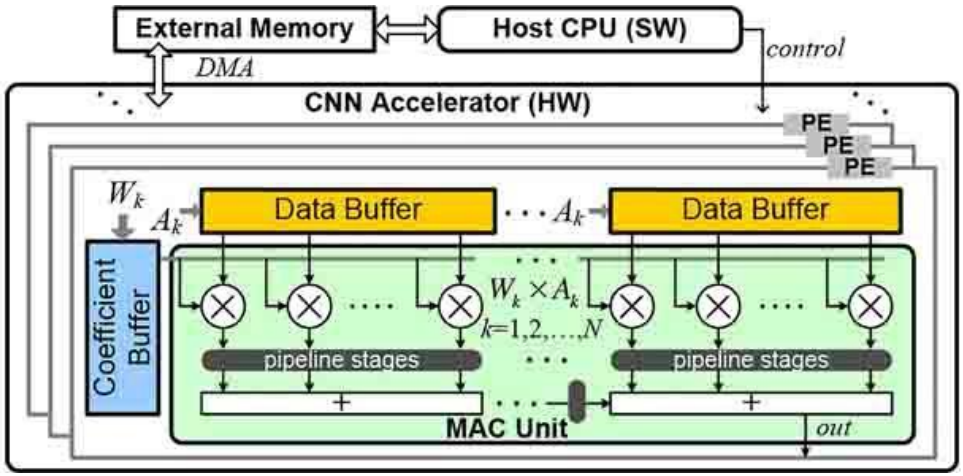
- Fixed Point Operation
- Problem 3

What is MAC?

- MAC is fundamental operation in DSP (P = A*B + C)
  - **Accumulator = Accumulator + (Operands1 * Operands2);**
  - MACs can be used in parallel way to accelerate AI application.



< MAC operation[1] >



< Structure of MAC units connected in parallel [2] >

- e.g.) Compute a Sum of 16 products $$Y = \sum_{i=0}^{15} A * B$$

[1] Gassoumi, Ismail, Lamjed Touil, and Bouraoui Ouni. "Design of efficient quantum Dot cellular automata (QCA) multiply accumulate (MAC) unit with power dissipation analysis." *IET Circuits, Devices & Systems* 13.4 (2019): 534-543.
[2] Tang, Song-Nien, and Yu-Shin Han. "A high-accuracy hardware-efficient multiply–accumulate (mac) unit based on dual-mode truncation error compensation for cnns." *Ieee Access* 8 (2020): 214716-214731.

Contents

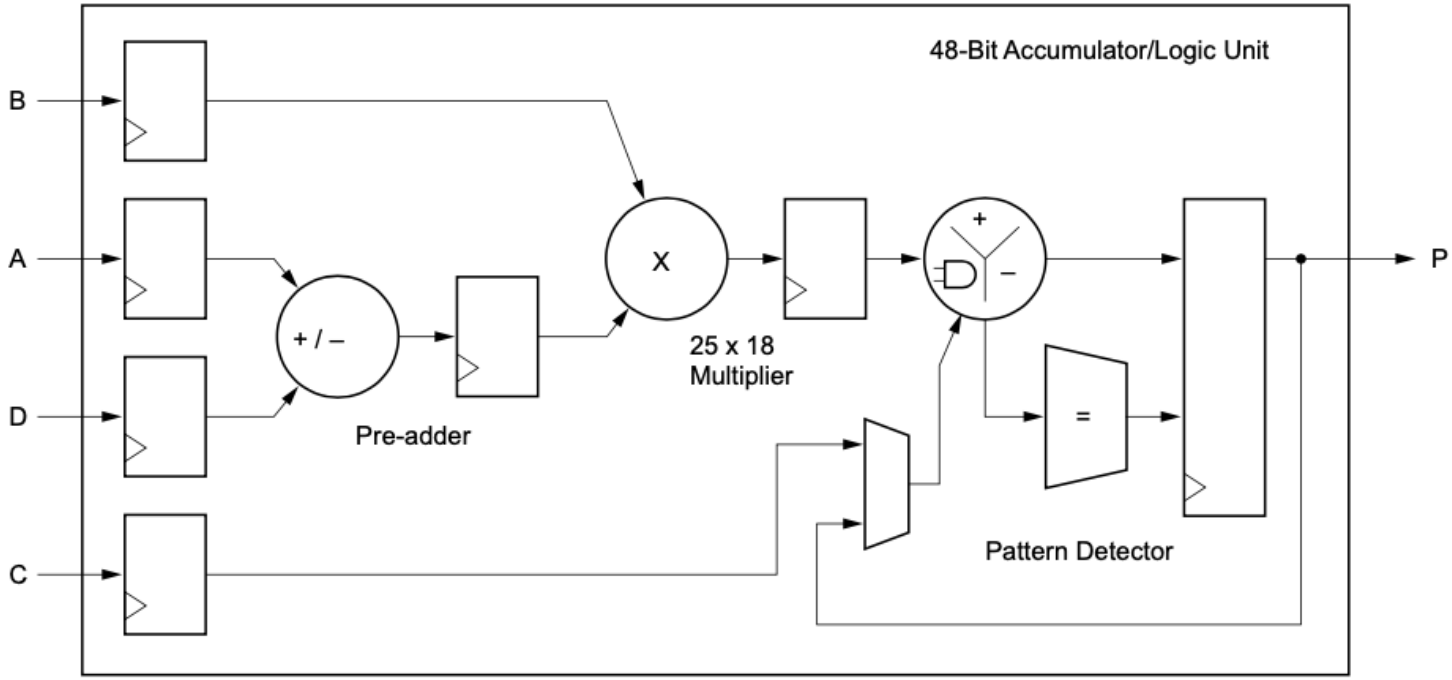
- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

What is a DSP?

- Digital Signal Processor (DSP)
  - It seems like ALU (Arithmetic Logic Unit) in CPU.
  - **It can operate as an adder, subtractor or multiplier.**



< DSP Block (DSP48) >

Contents

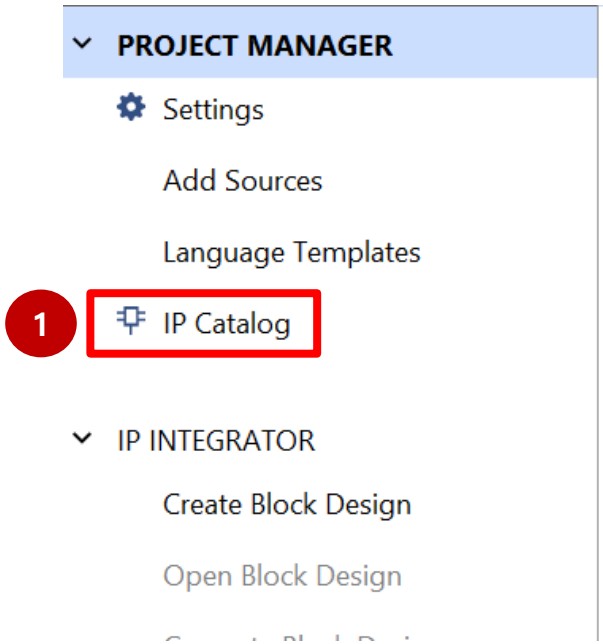
- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

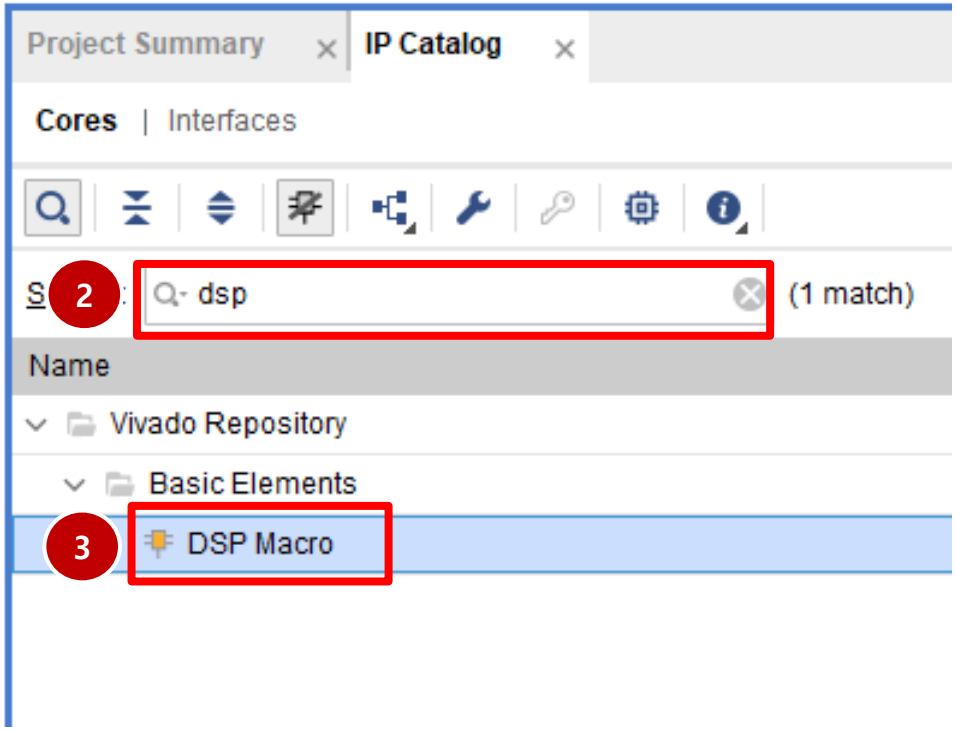
- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

How to generate a DSP in Vivado?

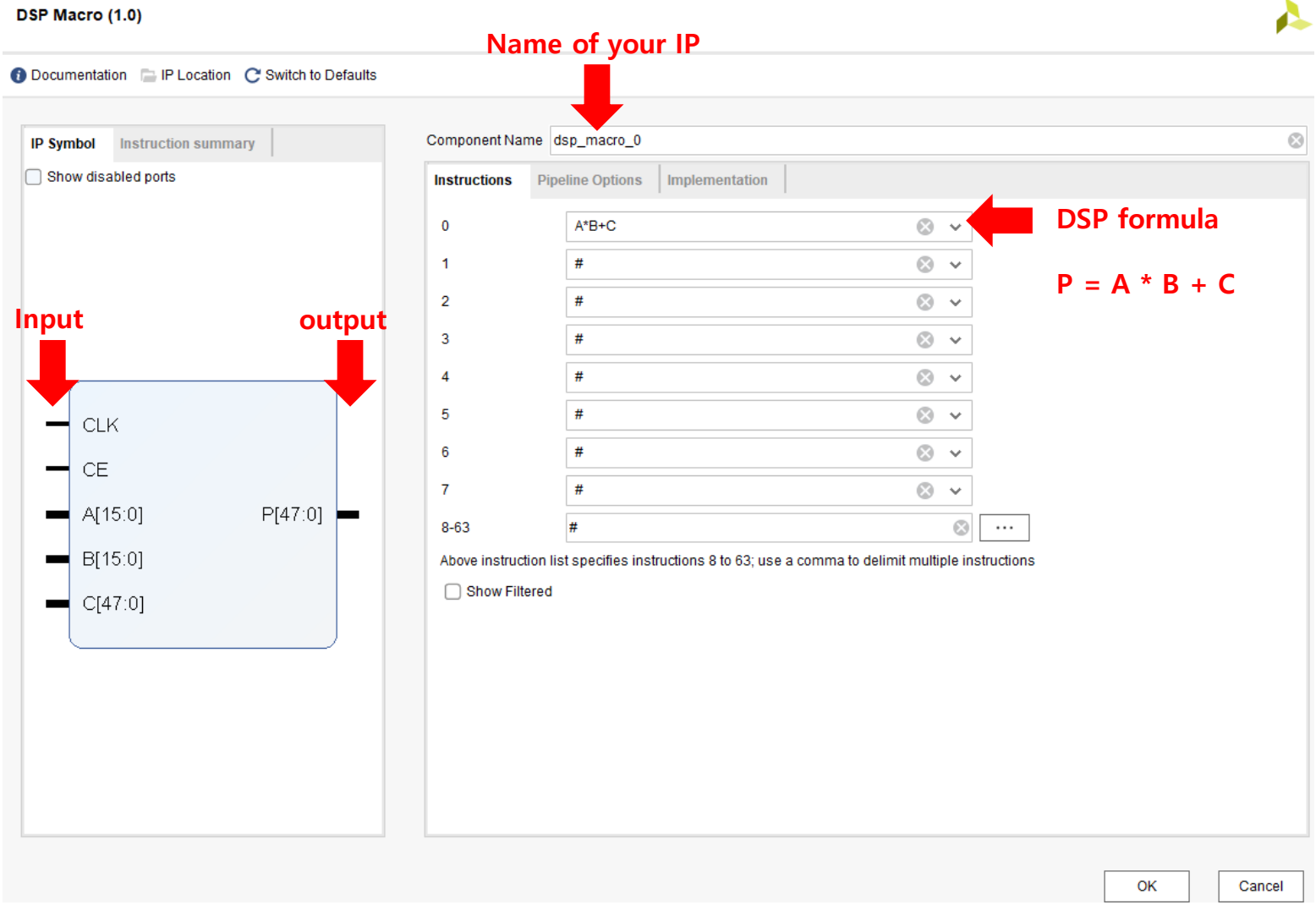
- Generate DSP in Vivado IP generator
  - Click “IP Catalog”
  - Search “DSP”
  - Choose “DSP48 Macro” and double click on it





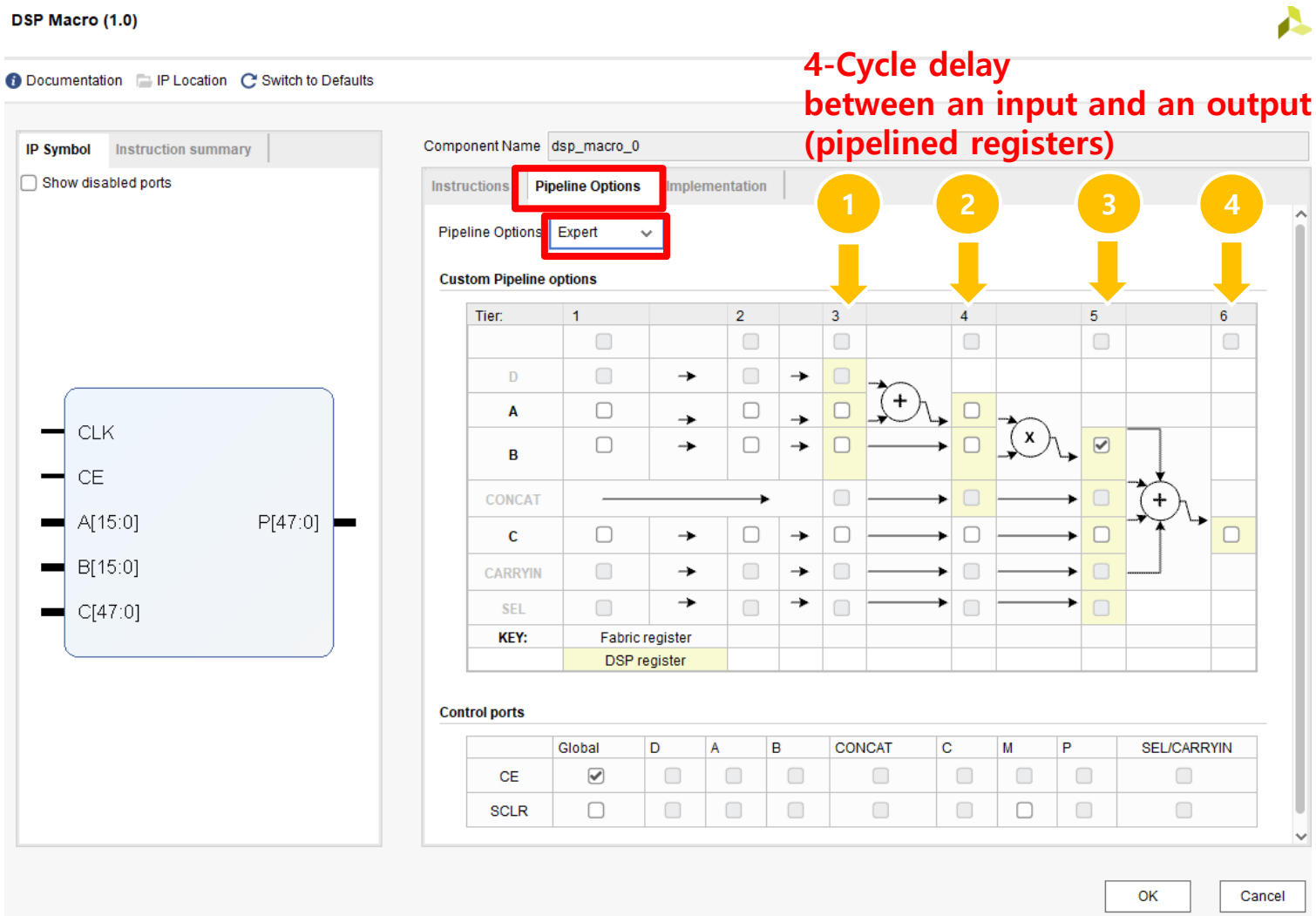
How to generate a DSP in Vivado?

DSP Configuration



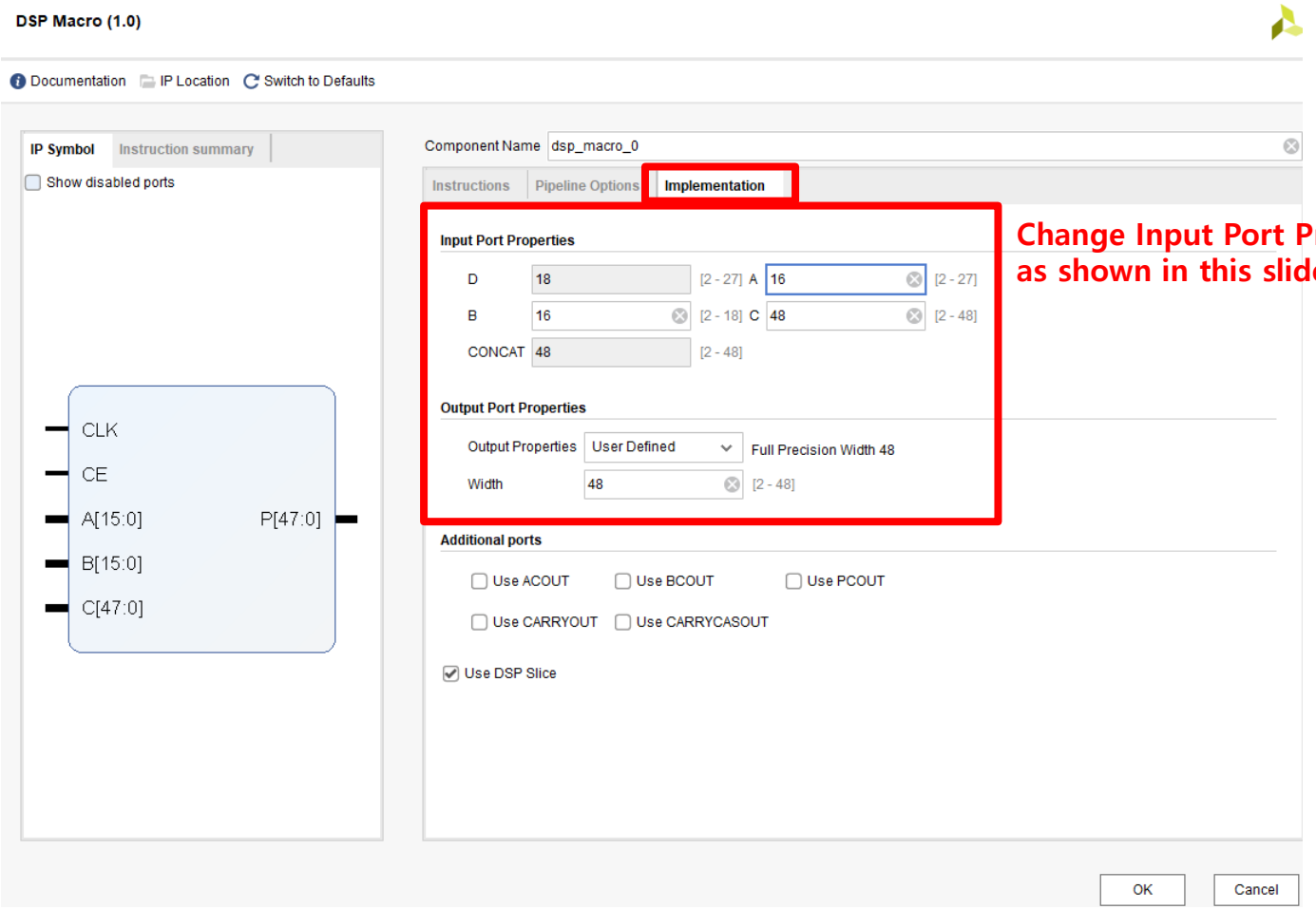
How to generate a DSP in Vivado?

DSP Configuration



How to generate a DSP in Vivado?

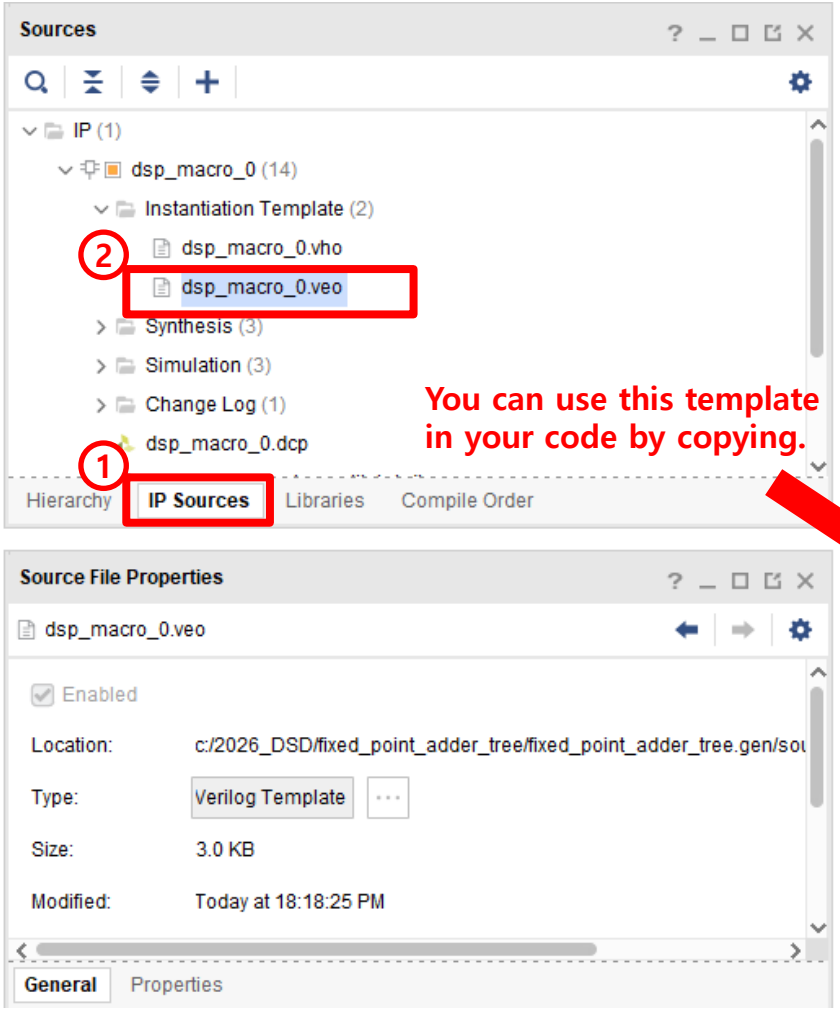
◈ DSP Configuration



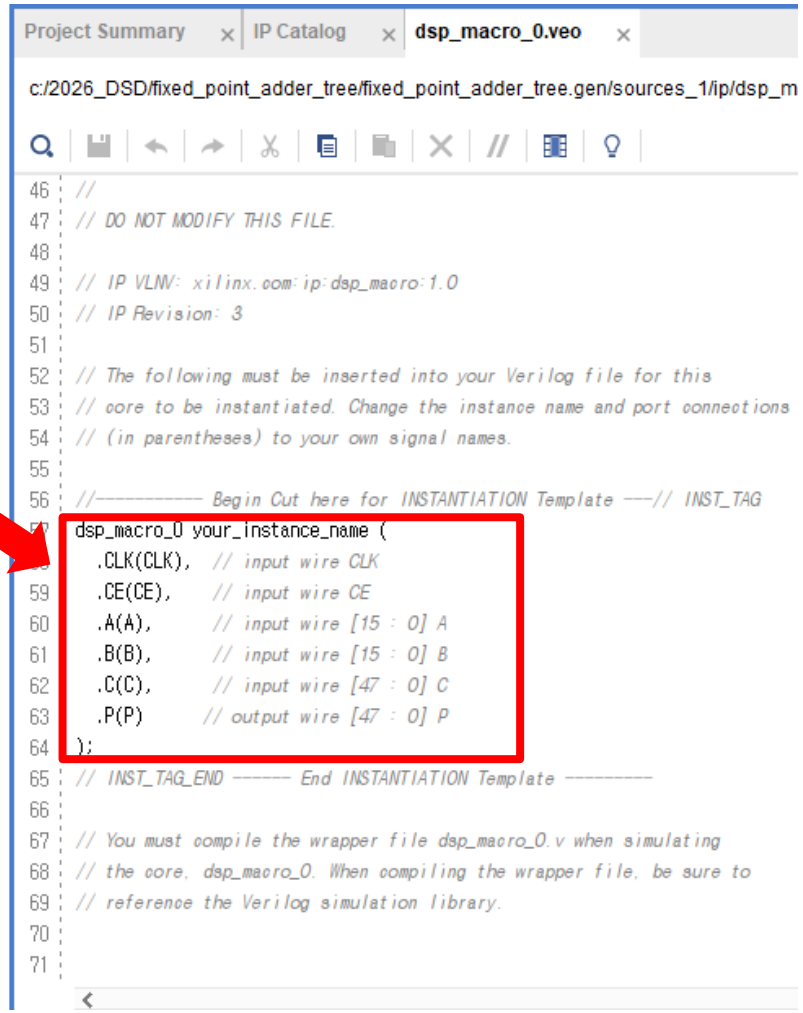
Change Input Port Properties as shown in this slides

How to generate a DSP in Vivado?

DSP code

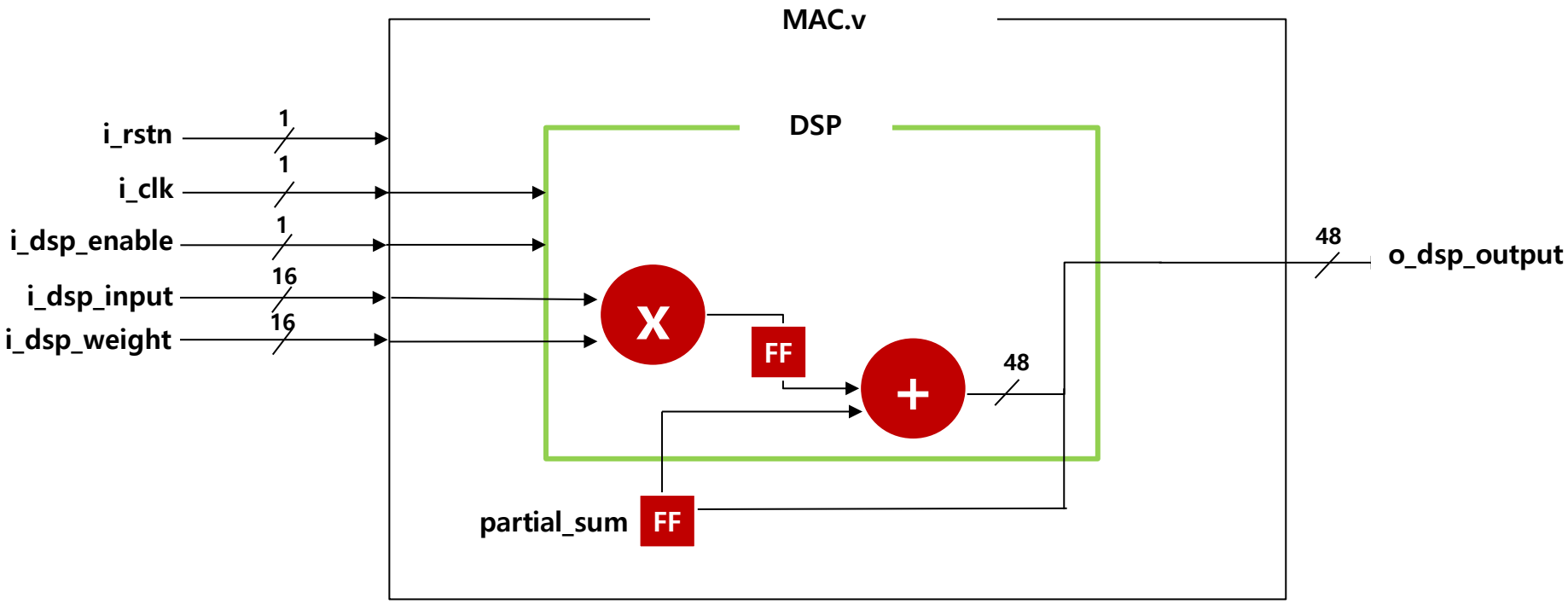


You can use this template in your code by copying.



Prob. 1 : MAC

- Let's make MAC operation code using DSP (MAC.v)
  - DSP takes previous output as an operand of accumulation
  - DSP uses global enable signal (dsp_enable).

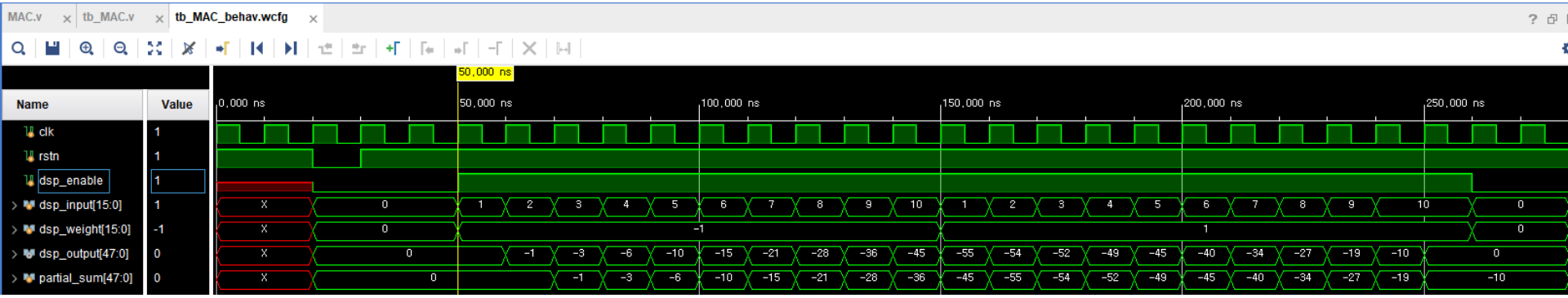


Prob. 1 – Testbench


```
//tb_MAC.v

`timescale 1ns / 1ps

module tb_MAC();

    reg clk;
    reg rstn;
    reg dsp_enable;
    reg signed [15:0] dsp_input;
    reg signed [15:0] dsp_weight;
    wire signed [47:0] dsp_output;

    initial begin
        clk = 1;
        forever
            #5 clk = ~clk;
    end

    //reset all system
    initial begin
        rstn = 1;
        #20 rstn = 0;
        dsp_enable = 0; dsp_input = 0; dsp_weight = 0;
        #10 rstn = 1;
        #10
        #10     dsp_enable = 1; dsp_input = 1; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 2; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 3; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 4; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 5; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 6; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 7; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 8; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 9; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 10; dsp_weight = -1;
        #10     dsp_enable = 1; dsp_input = 1; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 2; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 3; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 4; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 5; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 6; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 7; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 8; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 9; dsp_weight = 1;
        #10     dsp_enable = 1; dsp_input = 10; dsp_weight = 1;
        #20     dsp_enable = 0; dsp_input = 0; dsp_weight = 0;
    end


    MAC dut(
    .i_clk(clk),
    .i_rstn(rstn),
    .i_dsp_enable(dsp_enable),
    .i_dsp_input(dsp_input),
    .i_dsp_weight(dsp_weight),
    .o_dsp_output(dsp_output)
    );



endmodule
```

Prob. 1 – Waveform

- **Verify that the operator is functioning properly.**
  - Negative edge reset signal and enable signal
  - Proper results of output



Contents

- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

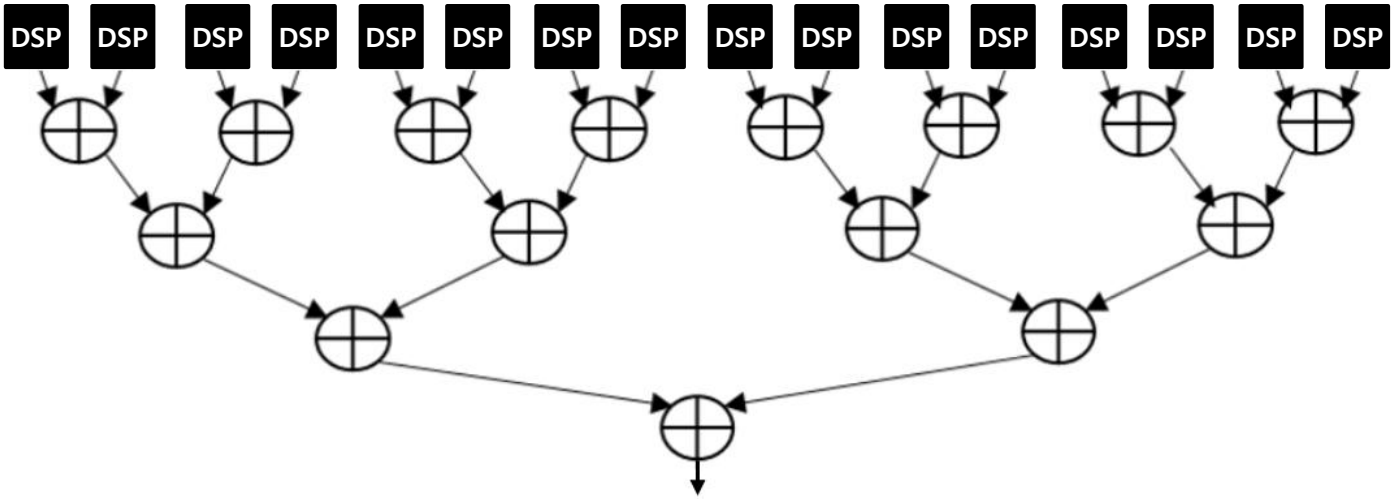
- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

What is an Adder Tree?

Adder Tree

- An **adder tree** is a hierarchical structure of adders designed for fast summation of multiple inputs.
- It performs additions in parallel, reducing computational delay compared to sequential addition.
- The more input data there is, the more effective the adder tree becomes.



Contents

- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

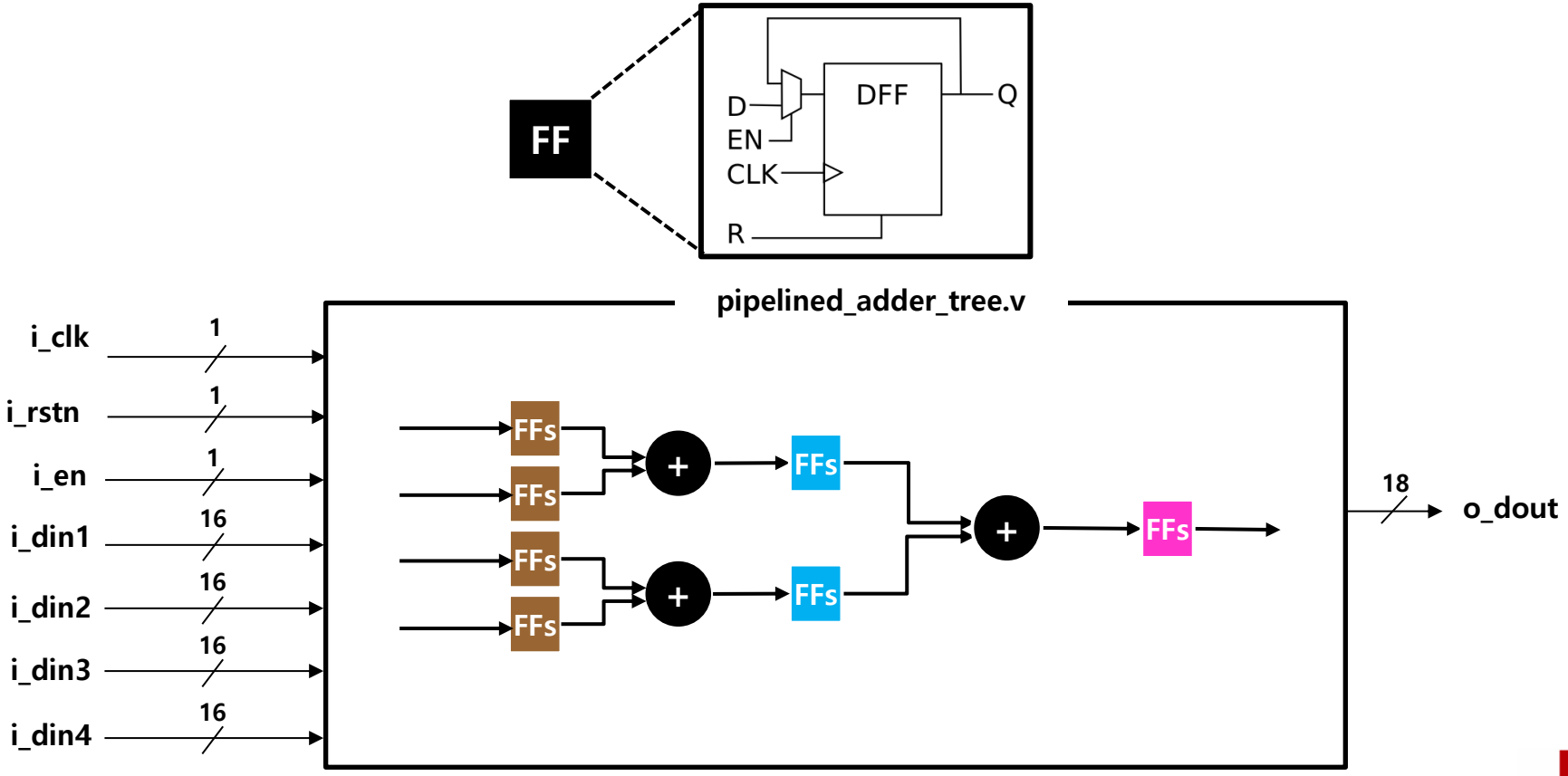
- Fixed Point Operation
- Problem 3

Prob. 2 : Pipelined Adder-Tree

Pipelined Adder Tree

▪Use the behavioral modeling

- Design a module that receives four 16-bit inputs (din1, din2, din3, din4) in each clk and one output signal as shown below.
- **All the flip-flops have "negative edge reset" and connected to an enable signal.**



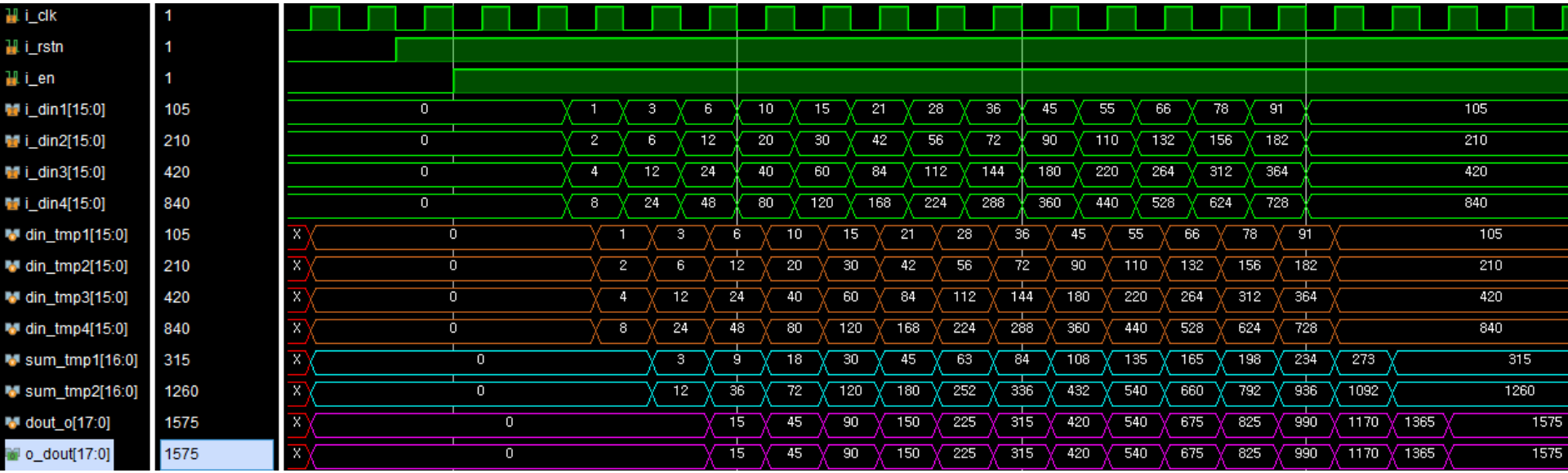
Prob. 2 – Testbench


```
// tb_pipelined_adder_tree.v

`timescale 1ns/1ps


module tb_pipelined_adder_tree();

    reg clk, rstn, en;
    reg [15:0] din1, din2, din3, din4;
    wire [17:0] result;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rstn = 1;
        #20 rstn = 0; din1 = 0; din2 = 0; din3 = 0; din4 = 0;
        #20 rstn = 1;
    end

    initial begin
        en = 0;
        #50 en = 1;
    end

    integer i;
    initial begin
        #60
        for (i = 0; i < 15; i = i + 1) begin
            din1 = din1 + i;
            din2 = din2 + i * 2;
            din3 = din3 + i * 4;
            din4 = din4 + i * 8;
            #10;
        end
        #50;
        $stop;
    end
```

```
    pipelined_adder_tree dut (
        .i_clk(clk),
        .i_rstn(rstn),
        .i_en(en),
        .i_din1(din1),
        .i_din2(din2),
        .i_din3(din3),
        .i_din4(din4),
        .o_dout(result)
    );
endmodule
```

Prob. 2 – Waveform

- **Verify that the operator is functioning properly.**
  - Negative edge reset signal and enable signal
  - Proper results of output



Contents

- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

Computing data format

Problem 3

Integer

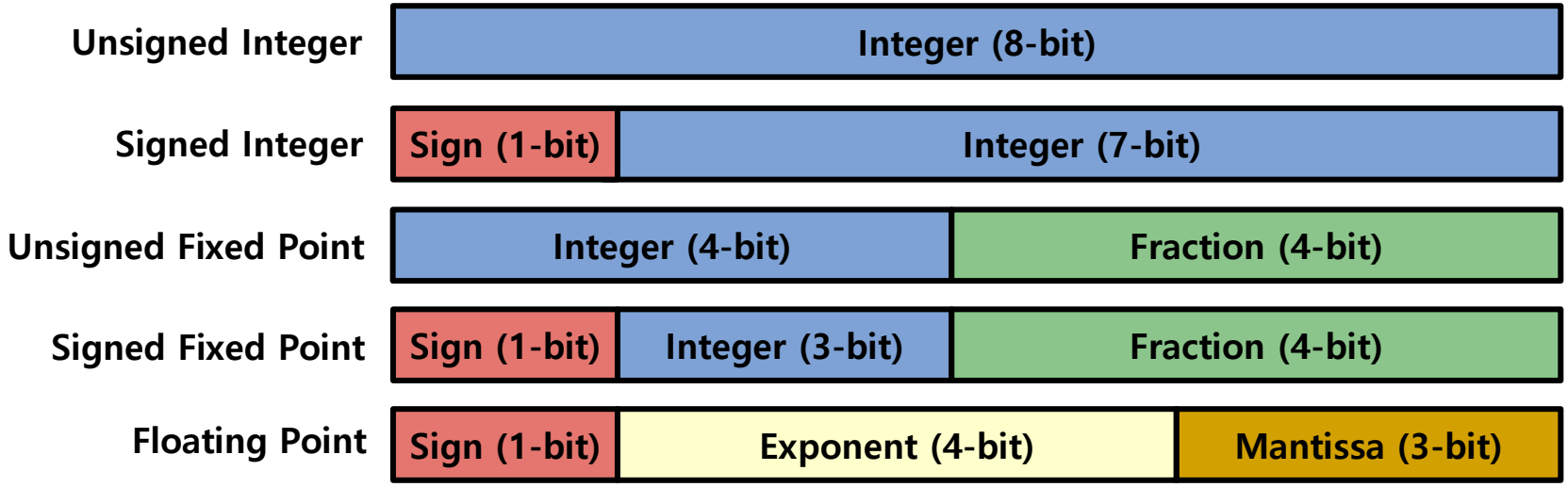
- The most basic data format for representing whole numbers.
- Can be unsigned or signed, and is simple to implement in hardware.

Fixed-Point

- A real-number representation that interprets bit patterns as integer and fractional parts.
- Efficient because it reuses integer hardware, but its range and precision are fixed.

Floating-Point

- A format that represents real numbers using sign, exponent, and mantissa
- Provides a very wide dynamic range, but requires more complex and costly hardware



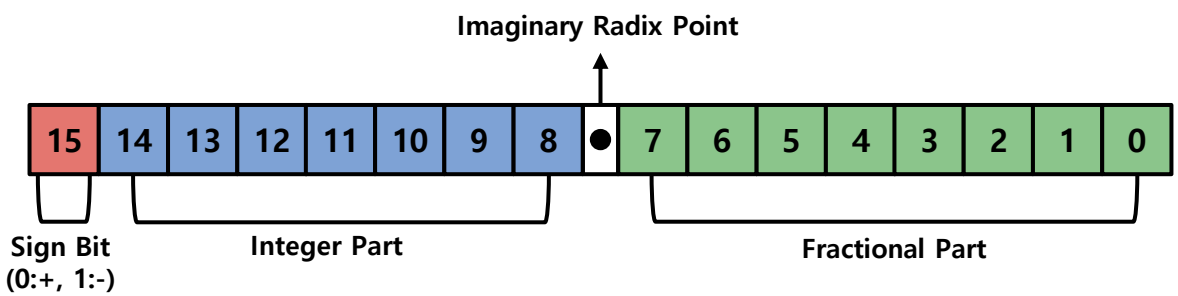
<8-bit numerical format>

Concepts of Fixed-Point

- A representation method that interprets **integer bit** patterns as if they were real numbers.
- Reuses existing integer ALUs (adders, multipliers) without complex additional circuitry.
- The **radix point** does not physically exist; it is only a logical convention defined by the designer.

Q8.8 Format

- Interprets a 16-bit register by splitting it into the upper 8 bits (integer part) and the lower 8 bits(fractional part).
- For a normal integer, the least significant bit (LSB) corresponds to $1(2^0)$.
- For Q8.8, the LSB corresponds to about $0.0039(2^{-8})$.
- Integers represent only whole-number steps of 1, while Q8.8 provides a resolution of about 0.0039 per step.



Fixed-point Operation

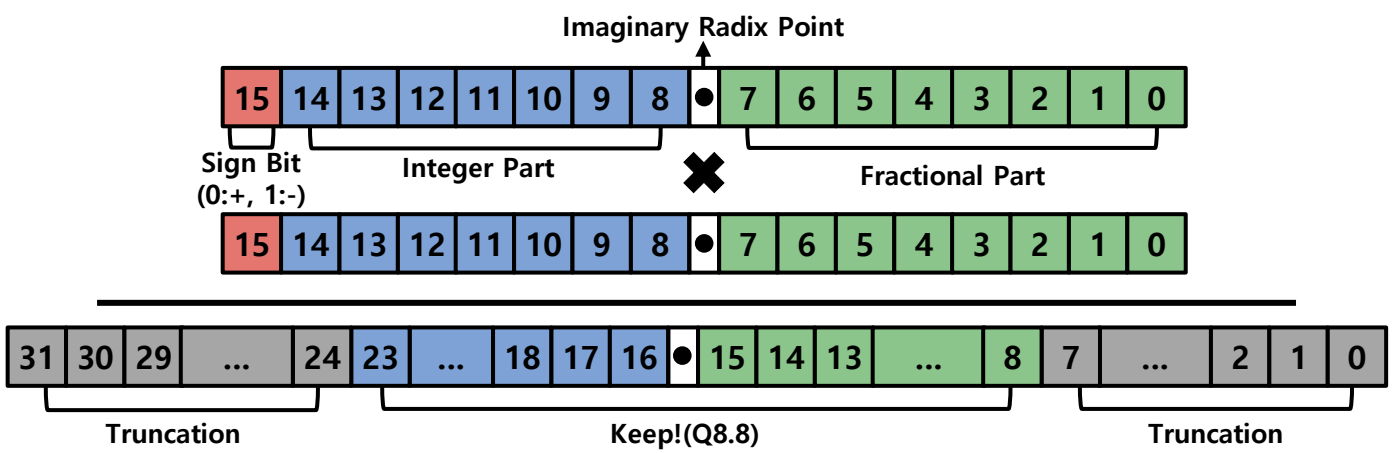
Problem 3

Addition and Subtraction

- Operations are performed between the same Q formats (e.g., Q8.8 + Q8.8 = Q8.8).
- A carry can push the result beyond the representable range.

Multiplication

- A 16-bit × 16-bit multiply produces a 32-bit result.
  - Multiplying two Q8.8 values yields a Q16.16 result.
- To match the target format, the result must be re-aligned back to Q8.8.
- Truncation
  - To output the 32-bit result as a 16-bit (Q8.8) value, the correct 16-bit slice must be extracted
  - Discard the lower 8 bits (loss of fractional precision)
  - Discard the upper 8 bits (assuming no large-integer overflow)
  - Extract bits [23:8] from the 32-bit result



Contents

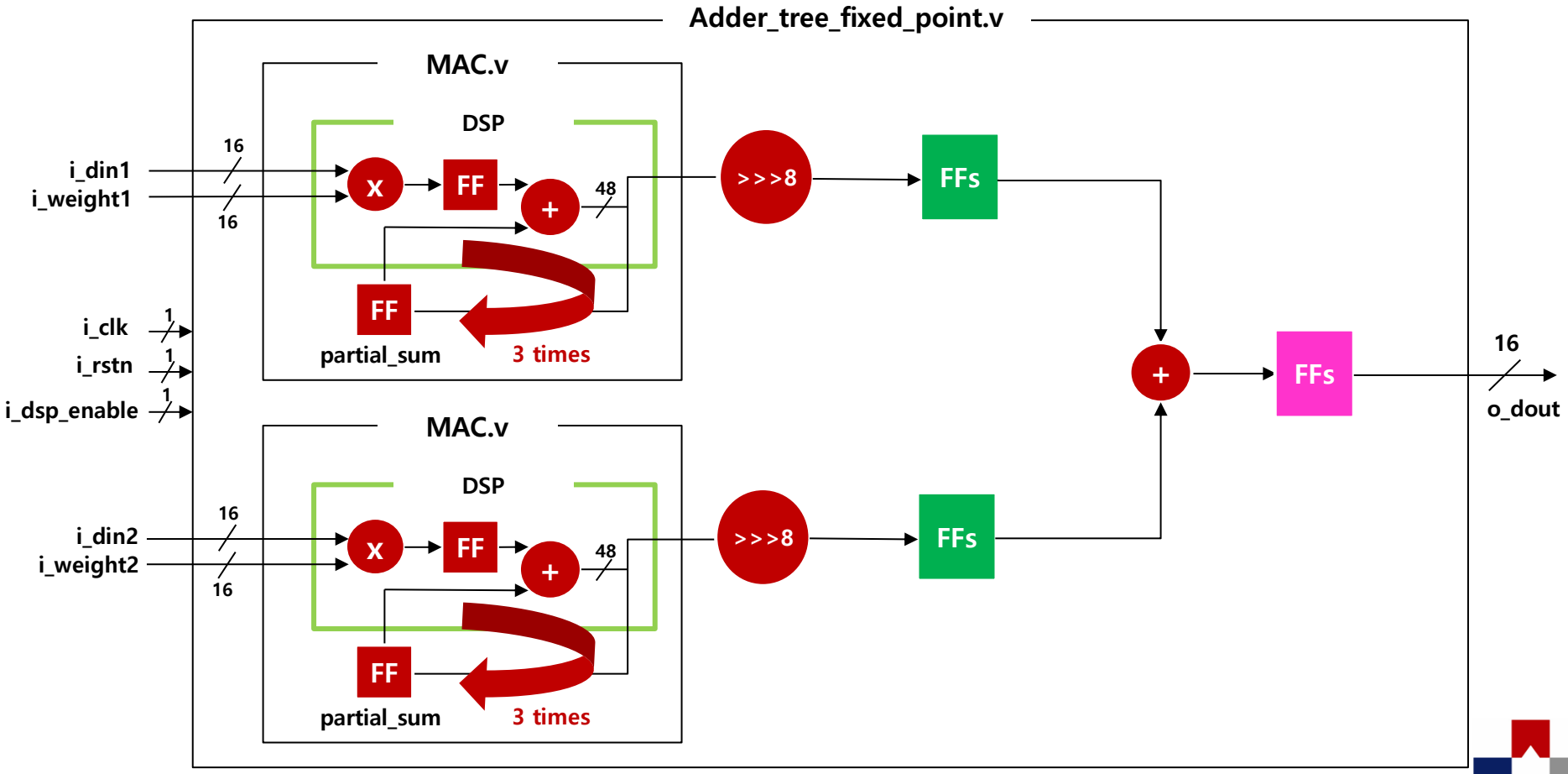
- Multiply-Accumulate (MAC)
- Digital Signal Processor (DSP)
- Problem 1

- Adder Tree
- Problem 2

- Fixed Point Operation
- Problem 3

Prob. 3 : Adder Tree: Fixed-Point

- **Design a fixed-point adder-tree module that combines two MAC outputs into one result.**
  - Reuse the previously designed MAC module as-is.
  - Inside each MAC module, **accumulation is performed three times.**
  - All inputs and the output are in Q8.8 fixed-point format.

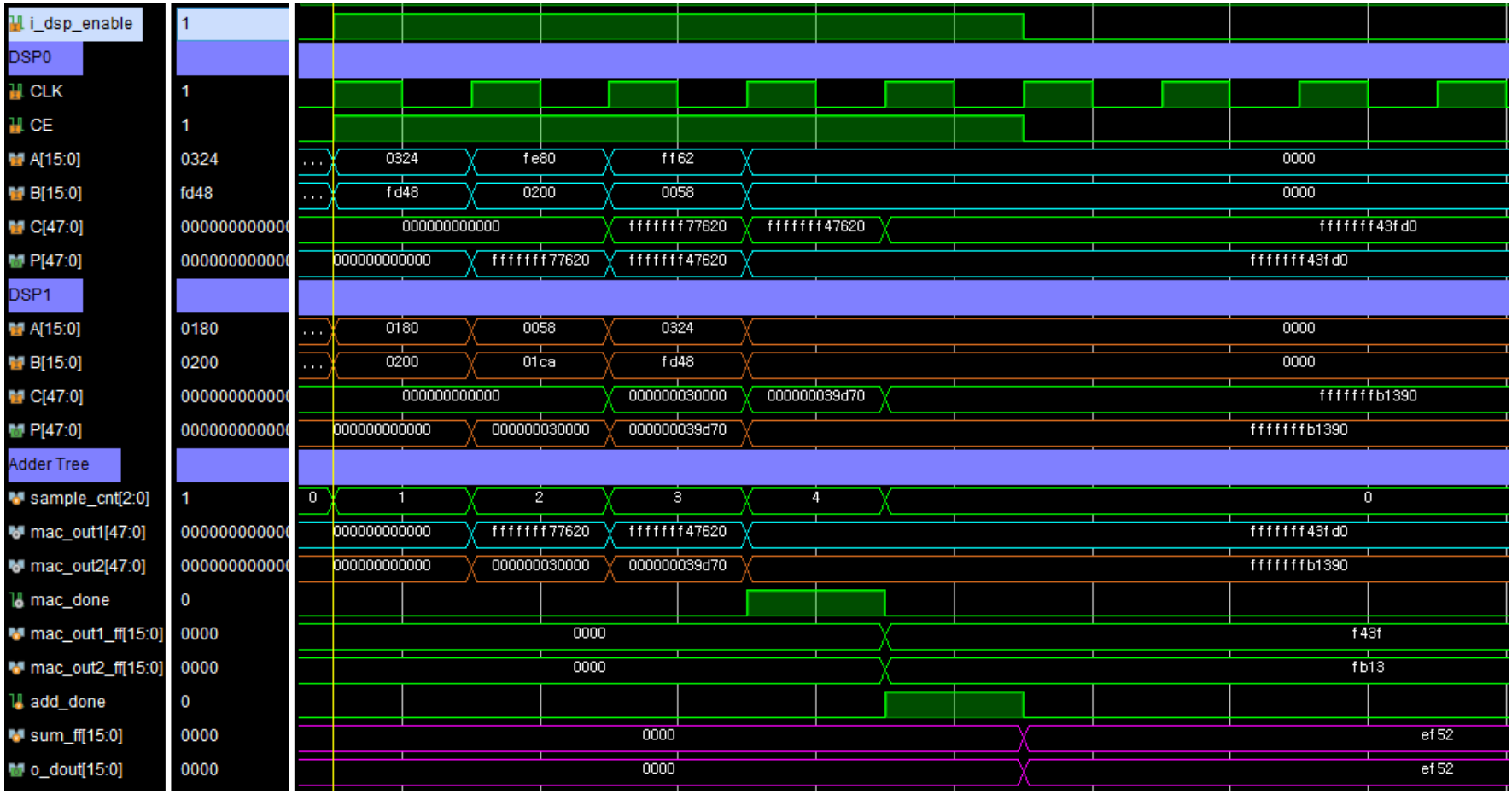


Prob. 3 – Testbench


```
// tb_Adder_tree_fixed_point.v

`timescale 1ns / 1ps

module tb_Adder_tree_fixed_point();

    reg clk;
    reg rstn;
    reg dsp_enable;
    reg signed [15:0] din1;
    reg signed [15:0] din2;
    reg signed [15:0] weight1;
    reg signed [15:0] weight2;
    wire signed [15:0] dout;

    Adder_tree_fixed_point dut (
        .i_clk(clk),
        .i_rstn(rstn),
        .i_dsp_enable(dsp_enable),
        .i_din1(din1),
        .i_din2(din2),
        .i_weight1(weight1),
        .i_weight2(weight2),
        .o_dout(dout)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rstn       = 1'b0;
        dsp_enable = 1'b0;
        din1       = 16'sd0;
        din2       = 16'sd0;
        weight1    = 16'sd0;
        weight2    = 16'sd0;
        #20;
        rstn = 1'b1;
        #5;
        dsp_enable = 1'b1;
        // cycle 1
        din1    = 16'h0324; // 3.1415926 (Q8.8: 3.140625)
        weight1 = 16'hFD48; // -2.7182818 (Q8.8: -2.71875)
        // exp product(hex): 16'hF776, exp(real): -8.5390625
        din2    = 16'h0180; // 1.5
        weight2 = 16'h0200; // 2.0
        // exp product(hex): 16'h0300, exp(real): 3.0
        #10;

        // cycle 2
        din1    = 16'hFE80; // -1.5
        weight1 = 16'h0200; // 2.0
        din2    = 16'h0058; // 0.3418556 (Q8.8: 0.34375)
        weight2 = 16'h01CA; // 1.789321 (Q8.8: 1.7890625)
        #10;

        // cycle 3
        din1    = 16'hFF62; // -0.6172839 (Q8.8: -0.6171875)
        weight1 = 16'h0058; // 0.3418556 (Q8.8: 0.34375)
        din2    = 16'h0324; // 3.1415926 (Q8.8: 3.140625)
        weight2 = 16'hFD48; // -2.7182818 (Q8.8: -2.71875)
        #10;

        din1    = 16'sd0;
        din2    = 16'sd0;
        weight1 = 16'sd0;
        weight2 = 16'sd0;
        #20;

        dsp_enable = 1'b0;
        // Expected answer: EF52
        #60;
        $finish;
    end

endmodule
```

전기정보공학과

SEOULTECH

Prob. 3 – Testbench

- **Verify that the operator is functioning properly.**
  - Negative edge reset signal and enable signal
  - Proper results of output